

universität wien

BACHELORARBEIT / BACHELOR'S THESIS

Titel der Bachelorarbeit / Title of the Bachelor's Thesis

„Enhancing ProcessIntel: Infrequent Inductive Mining, XES Data Integration, and Static Graph Visualization“

verfasst von / submitted by

Valeriia Kalinichenko

angestrebter akademischer Grad / in partial fulfilment of the requirements for the degree of

Bachelor of Science (BSc)

Wien, 2026 / Vienna, 2026

Studienkennzahl lt. Studienblatt / degree programme code as it appears on the student record sheet:	UA 033 521
Studienrichtung lt. Studienblatt / degree programme as it appears on the student record sheet:	Bachelorstudium Informatik
Betreut von / Supervisor:	Marian Lux, Dipl.-Ing. Dr.techn.

Abstract

This thesis focuses on extending and improving ProcessIntel, an open-source process mining application that is extended by multiple contributors. Maintaining it requires new functionality to be integrated consistently while remaining compatible, maintainable, and testable. The work combines process mining functionality with practical software challenges related to integration, adaptation, testing and visualization.

The thesis extends ProcessIntel with three main features. First, the Inductive Miner Infrequent algorithm from a previous student project was adapted to the current project architecture and integrated with the existing mining functionality. Second, support for the XES file format was integrated, including import functionality and conversion between XES and CSV file types in both directions. Third, an option for static graph layout visualization was implemented. When the option is activated, the layout of the currently shown graph is preserved and reused for filtering actions, making visual comparison easier for users.

The implemented functionality was evaluated using automated tests, manual user-interface testing, and comparison of the CSV to XES conversion with PM4Py. The work also identified limitations related to the static layout, conversion validation, and the processing of large event logs.

Overall, this thesis contributes to the continued development of ProcessIntel by integrating additional process mining functionality, improving visualization, and adapting existing implementations to the current software architecture.

Contents

Abstract i

1 Motivation 1
1.1 Goals of the Thesis . . . 1
1.1.1 Integrating the Inductive Miner Infrequent algorithm . . . 1
1.1.2 Fixing BPMN graph visualization behavior . . . 1
1.1.3 Integrating and improving XES import and conversion . . . 2
1.1.4 Evaluating XES conversion against PM4Py . . . 2
1.1.5 Adding static graph layout support for filtering . . . 2

2 Basics & Related Work 3
2.1 Foundations . . . 3
2.1.1 Event log . . . 3
2.1.2 Process Discovery . . . 4
2.1.3 Visualization . . . 4
2.2 Inductive Miner and Inductive Miner Infrequent . . . 5
2.3 XES file format . . . 5

3 Implementation 9
3.1 System Architecture . . . 9
3.2 XES Import and Format Conversion . . . 10
3.2.1 XES File Support . . . 10
3.2.2 Converters . . . 10
3.3 Integration of Inductive Miner Infrequent . . . 12
3.3.1 Mining Algorithm Integration . . . 12
3.3.2 View and Controller . . . 13
3.4 Visualization and Filtering Improvements . . . 14
3.4.1 BPMN Graph Creation for Empty Activity Sets . . . 14
3.4.2 Static Graph Layout for Filtering . . . 14

4 Evaluation and Discussion 17
4.1 Evaluation Methodology . . . 17
4.2 Evaluation of Implemented Features . . . 17
4.2.1 XES Import and Conversion . . . 17
4.2.2 Inductive Miner Infrequent . . . 18
4.2.3 BPMN Graph Edge Case . . . 18
4.2.4 Graph Layout Feature . . . 18

5 Conclusion and Future Work **21**
5.1 Conclusion 21
5.2 Limitations 21
5.3 Use of AI Tools 21
5.4 Lessons Learned 22
5.5 Future Work 22

Bibliography **23**

1 Motivation

Process mining is a valuable tool for any business, which provides end-to-end solutions and enables faster business decisions [12]. It can extract data from the company event logs, visualize current processes through process models, which allows customizing, comparing, processes as well as identifying inefficiencies in process.

ProcessIntel is an open-source process mining application developed and maintained by a nonprofit research center SWISDATA[1]. Unlike many other process mining tools, ProcessIntel is extended, fixed, tested, and validated mainly through collaboration of bachelor thesis students, with each contributor adding new features, improving existing functionality, and validating correctness. The application is available on the ProcessIntel website[2], allowing it to be accessed and evaluated by anyone without local installation.

The tool is developed in parallel by many students, which increases the need for project structure to be easily maintainable, expandable and follow the strict coding guidelines. Participating in multi-author, long-term collaboration is itself a central motivation for this thesis.

1.1 Goals of the Thesis

The scope and content of the improvements of this project were agreed with the supervisor at the beginning and included several goals.

1.1.1 Integrating the Inductive Miner Infrequent algorithm

The application already offered several mining algorithms for process discovery: Heuristic Miner, Fuzzy Miner, Inductive Miner, Alpha Miner, and Genetic Miner. Another student had developed a separate application that included the Inductive Miner Infrequent algorithm. This algorithm extends functionality of the standard Inductive Miner by introducing a filtering threshold for infrequent behavior in the event log. As a result, rare or exceptional behavior can be removed during process discovery, leading to cleaner and more readable process models. Integrating this algorithm into ProcessIntel gives users a way to handle noise and rare behavior in event logs.

1.1.2 Fixing BPMN graph visualization behavior

During the integration of the Inductive Miner Infrequent algorithm, an edge-case bug was discovered. When all activities were removed by filtering, the visualization of BPMN

[1] SWISDATA gGmbH, a nonprofit research and development organization: `https://www.swisdata.eu/`.
[2] ProcessIntel online process mining application: `https://processintel.org/`.

graph failed for both Inductive Miner and Inductive Miner Infrequent. The application tried to visualize activities even though all were supposed to be filtered out. This graph generation had to be fixed to keep the correct visualization.

1.1.3 Integrating and improving XES import and conversion

The file upload functionality originally supported only CSV file type which restricted the usage of the application. XES is an official IEEE standard for storing event logs and is commonly used by process mining tools because it provides a structured format with defined attributes and data types. Therefore, support for XES is necessary to improve compatibility with other process mining tools. The goal was to integrate the existing CSV to XES and XES to CSV converters from another student project into the current application. During the integration, a bug related to column-name handling in the converter also had to be fixed.

1.1.4 Evaluating XES conversion against PM4Py

Another goal was to evaluate the results of the CSV to XES conversion by comparing the converted files against the PM4Py Python library[3]. This comparison was intended to evaluate whether the conversion logic produced compatible output.

1.1.5 Adding static graph layout support for filtering

The application offers various toggles to adjust event filtering in the sidebar. However, when the filter values were changed, the application generated a new graph in which the positions of activities and edges also changed. This made it difficult to notice which activities had been filtered out. Therefore, a new feature had to be introduced to create the static positioning of graph, so that when events are removed it is visually easier to notice the differences.

[3]PM4Py (Process Mining for Python), an open-source Python library for process mining: `https://processintelligence.solutions/pm4py/`.

2 Basics & Related Work

2.1 Foundations

Almost every process that a business undergoes often leaves traces. In case of digital traces, they are usually system logs or records of what happened and when, this data can be loaded into a process mining tool and used to generate a process model. The model is based on observed process data and can provide a data-driven view of how the process was actually executed [14] [13].

Process mining features do not end here, if users have questions about a process, such as where repeated cycles occur, where additional time is spent, or where employee resources are poorly managed, process mining can help answer these questions through analysis and visualization [1]. After implementing improvements, users can check whether these changes affect the process behavior and lead to better results.

An important requirement of process mining is the structure and quality of the underlying data. It must be possible to identify what happened, which activity was performed, and when it occurred [14] [13].

2.1.1 Event log

The primary source of data for process mining is an event log. A data source may be anything which contains recorded data, such as an Excel file, a database table, or a transaction log [14]. However, such raw data is not automatically suitable for process mining. It first has to be filtered and transformed so that the data contained in the resulting event log relates to a single process [14].

An event log typically contains events that are grouped into cases. Each event records an activity that occurred as part of a specific process instance and may contain additional attributes such as timestamp, resource, or cost [14].

Table 2.1 shows a small example of an event log for an order-delivery process, it consists of two different cases with ids 1 and 2. The two cases represent two different executions of the same process. Case 1 shows a successful execution, it starts with *Order Created*, then *Package Packed*, *Shipped via Carrier*, and finally ends with *Delivered*. Case 2 shows a different path of the same process, it starts with *Order Created* and ends with *Order Cancelled*. Each row represents a single event. The *Activity* column describes what happened and *Timestamp* column when the event happened. By grouping the events by case ids and putting them in order this way trace can be constructed. [13]

Table 2.1: Example event log

Case ID	Activity	Timestamp
1	Order Created	2026-01-10 09:00
1	Package Packed	2026-01-10 09:15
1	Shipped via Carrier	2026-01-11 13:00
1	Delivered	2026-01-15 13:00
2	Order Created	2026-01-12 10:00
2	Order Cancelled	2026-01-12 13:00

2.1.2 Process Discovery

Process discovery is the fundamental task of process mining. It takes an event log as input and automatically constructs a process model that describes the behavior recorded in the log. It is the first step after extracting the data from the source systems. The algorithms like the Alpha algorithm or Inductive Miner analyze the event logs and generate a process model, such as a Petri net, Directly-Follows Graph (DFG), or BPMN diagram [13].

An important point is that different discovery algorithms use different approaches to construct a process model from an event log. For example, process discovery approaches can be divided into bottom-up and top-down approaches, which differ in how the process model is being constructed. Bottom-up approaches like the Alpha algorithm use constraints between individual activities, which can produce really dense, unstructured models which are harder to read. Top-down approaches, such as Inductive Miner, recursively divide the event log into smaller sublogs and combining resulting models into a process tree, which can create a cleaner, more structured visualizations. The choice of discovery algorithm matters because different algorithms can produce different process models from the same event log, which can affect both model structure and readability [13].

Another challenge in process discovery is finding a balance. The model should neither be 'overfit' (simply enumerate the observed traces) nor 'underfit' (allow behavior unrelated to what was observed) [14]. This is challenging because event logs are usually incomplete and contain no negative examples, so we cannot observe which behavior is impossible, and the four quality dimensions (recall, precision, generalization, and simplicity) often conflict [13].

Once the model is discovered, it provides a data-driven representation of the behavior recorded in the event log.

2.1.3 Visualization

Visualization plays an important role in process mining by supporting the exploration of process models and event-log behavior. Combining process mining with visual analytics helps users explore and understand large amounts of event log data and draw meaningful conclusions for it [6].

Visualization is not only used for displaying the discovered process model. It can also present additional process information such as bottlenecks and support different

perspective, such as performance, data, and resources

There are different visualization types. According to Eckhard et al. [1], the Directly-Follows Graph (DFG) was the most frequently used visualization type and is used to display the control-flow perspectives. Other visualization types are used to present other perspectives such as time, data, and resources. The choice of visualization is important for displaying and analyzing the particular aspect of process model.

2.2 Inductive Miner and Inductive Miner Infrequent

Inductive miner (IM) is a process discovery technique that follows a top-down algorithm approach and creates a process tree. The IM algorithm recursively splits the initial log into smaller sublogs. For each sublog, a Directly-Follows-Graph is created, and cuts are identified in the graph to determine how the log should be split. IM identifies cuts using four process operators: choice ($\times$), sequence ($\rightarrow$), parallel ($\wedge$), and redo-loop ($\circlearrowleft$). The resulting sublogs are mined recursively until a sublog contains a single activity [7]. An important advantage of IM is its ability to discover sound block-structured models [12].

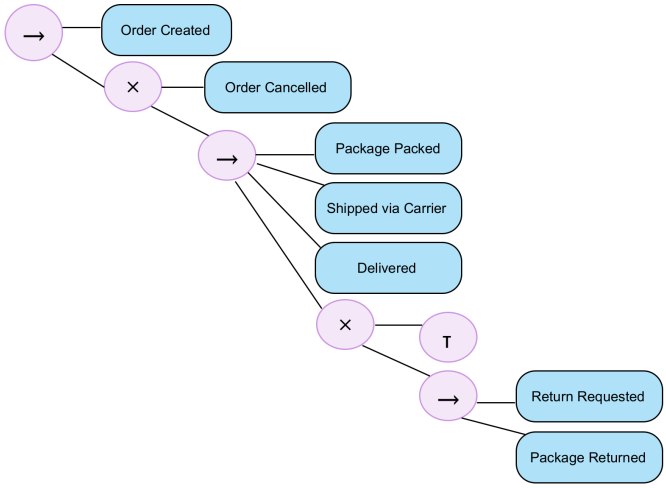
Inductive Miner Infrequent (IMf) extends Inductive Miner by filtering infrequent behavior during process discovery. The frequencies are ignored by IM but constantly used by IMf to distinguish frequent and infrequent behavior. Filtering is applied directly to the Directly-Follows-Graph, log splitting, base cases and empty traces [7]. A filtering threshold controls how much of "noisy" frequencies will be removed, and what is preserved in the discovered model. Increasing the filtering level removes more infrequent behavior, while lower filtering level preserves more of the behavior contained in the original event log. This provides control over the amount of infrequent behavior used during process discovery, which influences the resulting process tree.

To show the difference between Inductive Miner and Inductive Miner Infrequent, the event log from Table 2.1 can be extended with an additional rare trace. Assume that the log contains several traces of the variants *Order Created, Package Packed, Shipped via Carrier, Delivered* and *Order Created, Order Cancelled*, as well as one rare trace *Order Created, Package Packed, Shipped via Carrier, Delivered, Return Requested, Package Returned*.

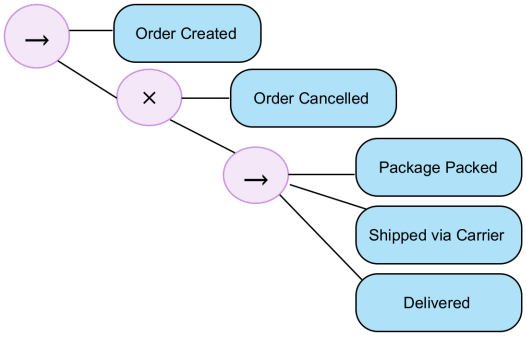
Figure 2.1b shows the process tree discovered by Inductive Miner Infrequent with a noise threshold of 0.2. In this case, the rare path trace *Order Created, Package Packed, Shipped via Carrier, Delivered, Return Requested, Package Returned* is filtered out as infrequent behavior, and the resulting process tree focuses on more frequent delivery and cancellation variants.

2.3 XES file format

XES (eXtensible Event Stream) [4] is an XML-based standard format for recording and sharing event logs between process mining tools [3]. XES file uses a hierarchical structure in which log contains traces, and each trace represents a sequence of events. Most XES files start with namespace declaration, which indicate that the XML elements belong to



(a) Process tree discovered by Inductive Miner.



(b) Process tree discovered by Inductive Miner Infrequent with a noise threshold of 0.2, where the infrequent package-return behavior is filtered out.

Figure 2.1: Comparison of IM and IMf on the same extended event log. The operator $\rightarrow$ denotes sequence, $\times$ denotes exclusive choice and τ denotes a silent activity representing no additional observable behavior.

the XES namespace (‘xmlns="http://www.xes-standard.org/"‘). XES supports typed attributes, including strings, dates, integers, floating-point values, collection types and other attribute types [5].

```
<log xmlns="http://www.xes-standard.org/"
     xes.version="1.0">

  <trace>
    <string key="concept:name" value="Case 1"/>

    <event>
      <string key="concept:name" value="Receive order"/>
```

```
      <date key="time:timestamp"
            value="2026-01-10T10:00:00+01:00"/>
    </event>

    <event>
      <string key="concept:name" value="Ship order"/>
      <date key="time:timestamp"
            value="2026-01-11T14:30:00+01:00"/>
    </event>
  </trace>

</log>
```

Listing 2.1: Example of a simplified XES event log

Listing 2.1 presents an example version of XES event log containing one trace with two events. The trace represents a single process instance identified as ‘Case 1‘. Each event contains a `concept:name` and `time:timestamp` attributes storing the performed activity name and time when it occurred.

3 Implementation

This chapter describes the implementation details of the project, including an overview of the system architecture, the newly implemented features, and bug fixes.

3.1 System Architecture

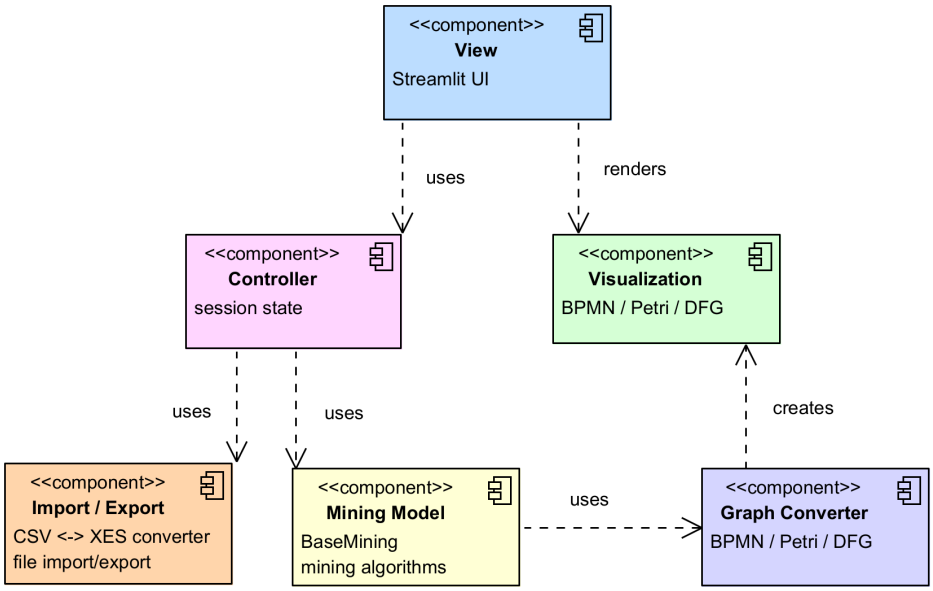


Figure 3.1: Component Diagram

The project has a modular architecture consisting of six main components. Figure 3.1 demonstrates the components and their relations.

The User Interface (`View`) is implemented with the Streamlit framework for Python [11]. It contains visual elements for uploading and exporting files, column selection, and mining algorithm selection.

The `Controller` component coordinates the interaction between the user interface and the application models. It handles user input, Streamlit session state, and errors. This project contains several controllers, such as `HomeController`, `ColumnSelectionController`, `ExportController`, and `BaseAlgorithmController`.

The `Mining model` component contains the implemented algorithms for process mining: Heuristic Miner, Fuzzy Miner, Inductive Miner, Alpha Miner, Genetic Miner and Inductive

Miner Infrequent. All the mining algorithms directly or indirectly inherit from the `BaseMining` base class, which itself extends the abstract `MiningInterface`.

The `Graph Converter` is a part of mining to visualization pipeline. It converts mining algorithm results into BPMN / Petri-net / Directly-Follows Graph(DFG).

The `Import / Export` component handles CSV and XES files. It is responsible for file import and export operations as well as conversion between two formats. The `Visualization` component contains the graph classes used to represent process mining results. The application currently supports BPMN, Petri net, and DFG graph representations.

The simplified mining flow looks like this:

User → Streamlit View → Controller → Mining Model → Graph Converter → Visualization → Streamlit View

3.2 XES Import and Format Conversion

The goal was to integrate XES file support and file conversion from XES to CSV and vice versa. This functionality had already been implemented in a previous student project, but it was part of a separate application and had not yet been integrated into the current ProcessIntel project.

3.2.1 XES File Support

The first step was the integration of XES file type support. These changes were made to support XES file format:

- The `.xes` file extension was added to `import_file_types_mapping`.
- Column prediction configuration was added for values imported from XES files.
- The `config.py` configuration was extended to support the XES file format.
- `ImportOperations` class was extended with `read_xes` function to read XES file and return an `ElementTree`.
- `HomeController` class was adapted to support XES file and process the created `ElementTree`.

3.2.2 Converters

The second step was the integration of the converters. The `CsvToXesConverter` and `XesToCsvConverter` were added for transforming CSV/XES event logs into XES/CSV format.

CSV To XES Converter

The `CsvToXesConverter` uses a `DataFrame` and three selected columns: case ID, activity and timestamp. It creates a tuple containing the XES XML tree, a summary of the converted data, and the number of rows removed because of missing values in the required columns. The summary information is later used for the data preview in the view. The input is validated for missing, duplicated, or invalid column selections. The selected CSV columns are renamed according to the corresponding XES attribute names, and timestamps are checked and formatted before the XES tree is created.

One of the tasks was to improve the XES converter. The existing implementation produced an incorrect XES file in a specific edge case. If the CSV file already contained a column named `concept:name`, but another column was selected as the activity column, both columns could result in the same XES attribute name and then be merged.

The `_get_original_name` method solved this problem by renaming the existing column using the `original:` prefix. If the resulting name already exists, a numeric suffix is added to create a unique name. For example, the resulting attribute names can be `original:concept:name:1`.

```
def _get_original_name(self, column_name: str, existing_columns: list[
    str]) -> str:
    original_name = f"original:{column_name}"
    if original_name not in existing_columns:
        return original_name
    counter = 1
    while f"{original_name}:{counter}" in existing_columns:
        counter += 1
    return f"{original_name}:{counter}"
```

Listing 3.1: Creating Original Column Names

After the attributes have been validated, renamed, and formatted, the XES tree is created.

XES To CSV Converter

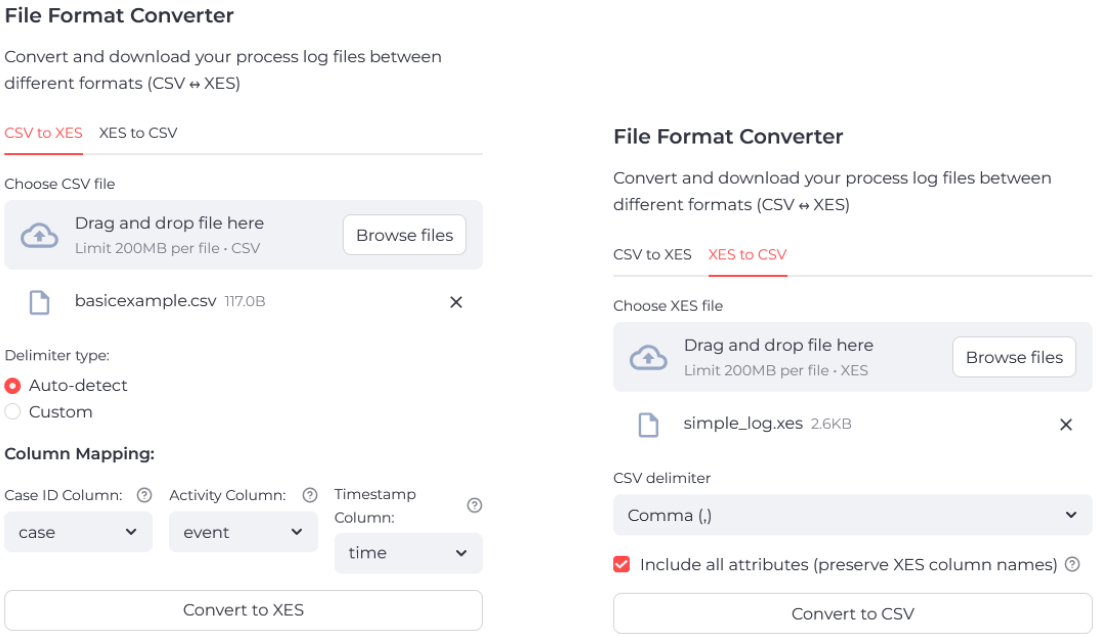
XES to CSV conversion is done by reading the XES file into an `ElementTree` and transforming its trace and event attributes into a pandas [8] `DataFrame`. The converter provides the option either to preserve all available attributes or to keep only the essential attributes: activity, case ID, and timestamp.

View and Controller

In `HomeView`, display functionality was added to render the converter user interface. The user can select between automatic and custom delimiter detection and configure the required column mappings.

The `HomeController` was extended to manage the conversion process and to predict suitable case ID, activity, and timestamp columns using `PredictionModel`.

After successfully uploading a file, the converters provide different configuration options. Figure 3.2 shows both conversion interfaces.



(a) CSV-to-XES Conversion Interface

(b) XES-to-CSV Conversion Interface

Figure 3.2: File converters.

After clicking the Convert to CSV or Convert to XES button, a Data Summary and Data Preview are displayed. The summary provides a short overview of the number of columns, cases, activities, and events. For CSV to XES conversion, the preview displays the first 10 events in tabular form. Figure 3.3 shows both conversion preview interfaces.

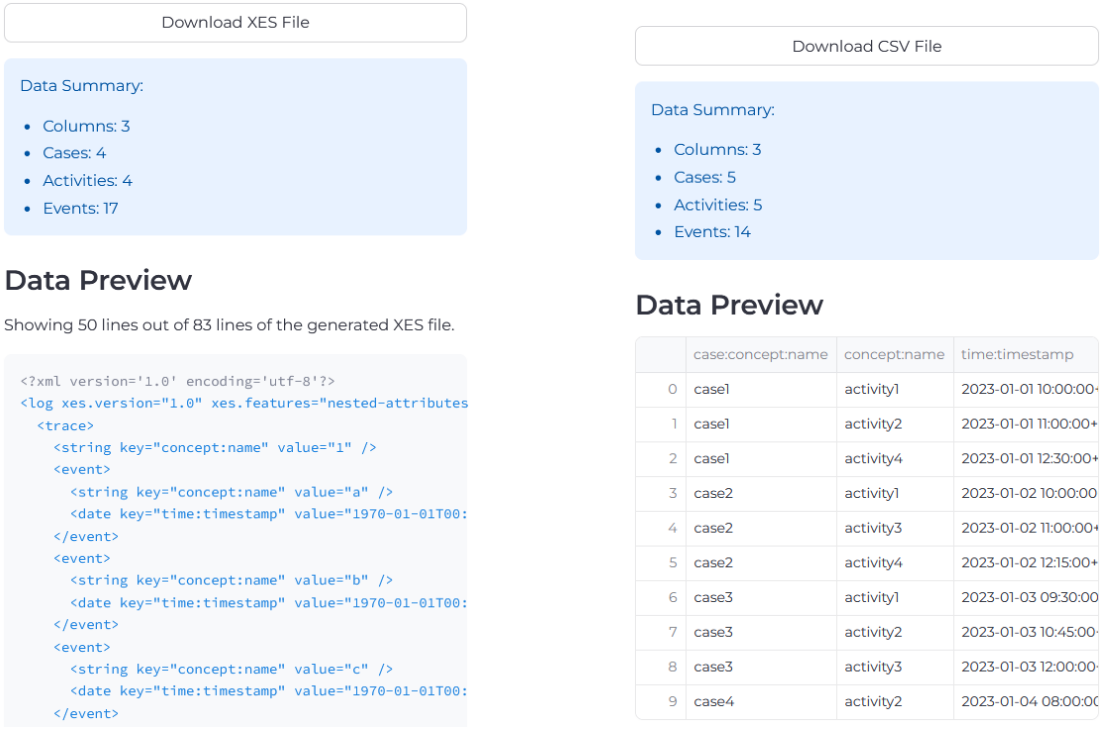
3.3 Integration of Inductive Miner Infrequent

The Inductive Miner Infrequent algorithm already existed in another student's project[1]. It had to be adapted to the current project architecture and integrated alongside the other mining algorithms.

3.3.1 Mining Algorithm Integration

The `InductiveMiningInfrequent` class was added to the mining algorithm component. It extends the existing `InductiveMining` class and reuses the common functionality of Inductive Miner while adding IMf specific behavior. The algorithm mappings in `config.py` were extended with Inductive Miner Infrequent route.

[1]Previous ProcessIntel student project implementing Inductive Miner Infrequent: `https://github.com/zaker1998/ProcessMiningVisualization_SS25_Dussaliyev_/tree/master`.



(a) CSV to XES Overview (b) XES to CSV Overview

Figure 3.3: Data Preview and Overview for converted data.

The algorithm introduces a noise threshold used to control the filtering of infrequent behavior. The default noise threshold parameter is set to 0.2, as this is the commonly used default configuration of Inductive Miner Infrequent introduced by Leemans et al. [7]. The specific algorithm behavior is implemented in `splits_imf.py`. It contains the sequence, parallel, exclusive, and loop split operations, as well as additional checks used during the recursive mining procedure.

3.3.2 View and Controller

To integrate the algorithm into the user interface, the `InductiveMinerInfrequentController` and `InductiveMiningInfrequentView` were added. The Controller and View extend the Inductive Miner View and Controller classes to reuse the functionality. In the View a Noise Threshold slider is introduced and short Guidance section was added to give short instruction points about threshold filtering. Figure 3.4 demonstrates the integrated IMf algorithm with its corresponding threshold.

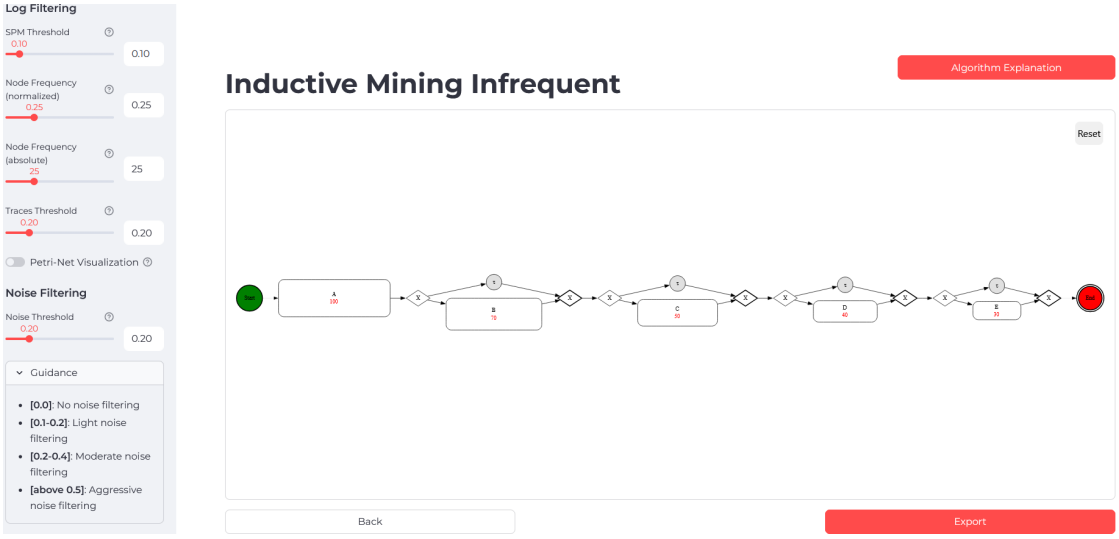


Figure 3.4: Inductive Miner Infrequent Interface

3.4 Visualization and Filtering Improvements

3.4.1 BPMN Graph Creation for Empty Activity Sets

A bug happened during BPMN Graph creation for the Inductive Miner algorithms (IM and IMf). When all activities were filtered out (SPM metric is set to 1.0), the system threw an error instead of handling this edge case.

The issue was handled in the `BPMNConverter`. Previously, when no filtered events were available, the converter did not correctly handle the empty activity set, which resulted in an error during graph creation. The implementation was changed by adding `_build_empty_graph` method. This method created an empty `BPMNGraph`, adds start Node and end Node, and finally creates an Edge between them.

```
    @staticmethod
    def _build_empty_graph(data: InductiveBPMNData) -> BPMNGraph:
        graph = BPMNGraph(data.filtered_events)
        graph.add_start_node()
        graph.add_end_node()
        graph.add_edge("Start", "End", None)
        return graph
```

Listing 3.2: Empty BPMN Graph creation method

3.4.2 Static Graph Layout for Filtering

When a user changes filter parameters, a new graph is generated. The positioning of nodes and edges change, which makes visual comparison between different filtering states more difficult. The task was to create a toggle that preserved the graph layout while filters are changed.

When the *Static Graph Layout* toggle is enabled, the current graph is stored in the Streamlit session state and used as a reference graph. When new filter parameters are applied, a new graph is generated and compared with the stored graph. Nodes and edges that are no longer present in the newly generated graph are marked as invisible, instead of being removed. This allows them to remain part of the graph structure used for layout calculation while no longer being displayed.

The graph visualization is generated using Graphviz [2]. To make nodes and edges invisible while preserving the existing graph structure, their visibility was modified in the generated Graphviz source.

The `Node` class was extended with the new field `attributes` to store and update attributes of a node. `BaseGraph` class was extended to keep track of invisible nodes and edges and to modify their visibility in the generated Graphviz source.

Making nodes invisible was done by adding another node declaration with `[style="invis"];` at the end in Graphviz source. It would look like this:

```
"A" [label="5"];
"A" [style="invis"];
```

Edges required a different approach. Initially, the Graphviz graph was not configured as a strict graph. Thus, adding another declaration for the edge with an invisibility attribute could create a separate invisible edge while the original visible edge remained. To solve this problem, the graph was changed to a strict directed graph. It meant that multiple edges between the same source and target are not allowed. As a result, an additional DOT declaration with `[style="invis"];` attribute applied to the already existing edge instead of creating a parallel edge, which is shown in Listing 3.3. This allows the edge to remain part of the graph structure while no longer being displayed.

```
    def _make_invisible_edge_lines(self) -> list[str]:
        lines = []
        for source_id, target_id in self._invisible_edge_ids:
            escaped_source_id = self._get_escaped_node_id(source_id)
            escaped_target_id = self._get_escaped_node_id(target_id)
            lines.append(
                f'\t"{escaped_source_id}" -> "{escaped_target_id}" [style="invis"];'
            )

        return lines
```

Listing 3.3: Editing DOT declarations for invisible edges

The main static layout logic is implemented in `_apply_fixed_graph_position` method. The reference graph is stored in session state, it gets compared to the newly generated graph. Missing nodes and edges are marked as invisible, while nodes that remain visible are updated with the information from the newly generated graph. Listing 3.4 shows the core logic of this process.

```
fixed_graph = st.session_state.get("graph_fixed_position")

fixed_graph.set_invisible_nodes(graph.get_node_ids())
fixed_graph.update_visible_nodes(graph.get_nodes())
```

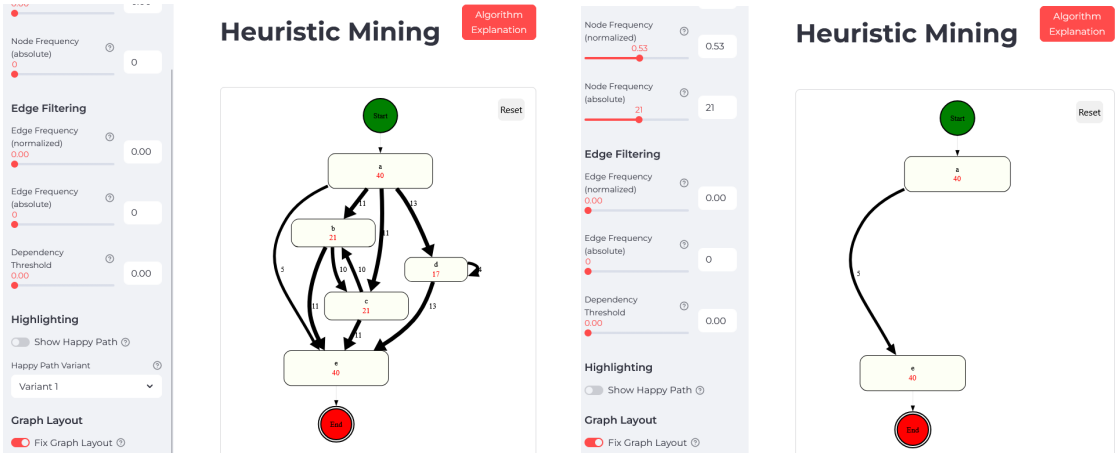
```
5 fixed_graph.set_invisible_edges(graph.get_edges())
6
7 return fixed_graph
```

Listing 3.4: Applying the static graph layout

The toggle is automatically disabled when the new graph contains nodes or edges that are not present in the saved reference graph. In this case, the reference graph can no longer preserve the full graph structure and a new layout has to be regenerated.

The static graph layout is currently supported for the DFG-based algorithms: Heuristic Miner and Fuzzy Miner.

Figure 3.5 shows the graph before and after the node filtering with the `Static Graph Layout` toggle enabled. The node frequency (normalized) was set to `0.53`, causing several nodes and edges to be filtered out, while the remaining graph elements preserved the original positions.



(a) no filter applied (b) filter applied

Figure 3.5: Static Graph Layout

4 Evaluation and Discussion

4.1 Evaluation Methodology

The evaluation methodology consisted of several stages, depending on the feature being evaluated. The usual process started with implementing the code and manually testing the functionality through the user interface using test files. The next step was writing unit tests, followed by creating a pull request where the supervisor reviewed the implementation for correctness, compliance with the existing architecture, clean integration, and docstring conventions. In most cases, the implementation required another round of improvements in order to fit smoothly into the project requirements. The last steps were repeated over several review cycles until not only correctness but also required project standards were achieved.

4.2 Evaluation of Implemented Features

4.2.1 XES Import and Conversion

The test cases from the previous XES implementation[1] were adapted to the current ProcessIntel architecture. Additional tests were introduced for functionality that was not covered by the previous implementation. These include preserving all XES attributes or selecting only essential attributes during XES to CSV conversion, validating the CSV to XES conversion result, and testing for the `concept:name` attribute merging issue.

To evaluate the correctness of the implemented fix for column-merging bug, the `BPI Challenge 2013` [10] dataset was used. The conversion was firstly tested manually through by visually inspecting the converted CSV columns. Also, a unit test was implemented to simulate behavior of the BPI Challenge 2013 dataset by containing an existing `concept:name` column.

One of the goals of this thesis was to test CSV to XES conversion for correctness by comparing its output with `PM4Py` [9]. The same CSV input was converted using both ProcessIntel `CsvToXesConverter` and the PM4Py converters. The resulting XES logs were compared based on case identifiers, activity values, timestamps, and total number of events. The comparison showed equivalent results for the evaluated input. The implemented CSV to XES validation is currently stored only in the `validate-csv-to-xes-conversion` branch of author's fork and has not been integrated into upstream ProcessIntel repository.

[1] Previous ProcessIntel student project implementing XES Import, Conversion and Inductive Miner Infrequent: `https://github.com/zaker1998/ProcessMiningVisualization_SS25_Dussaliyev_/tree/master`.

4.2.2 Inductive Miner Infrequent

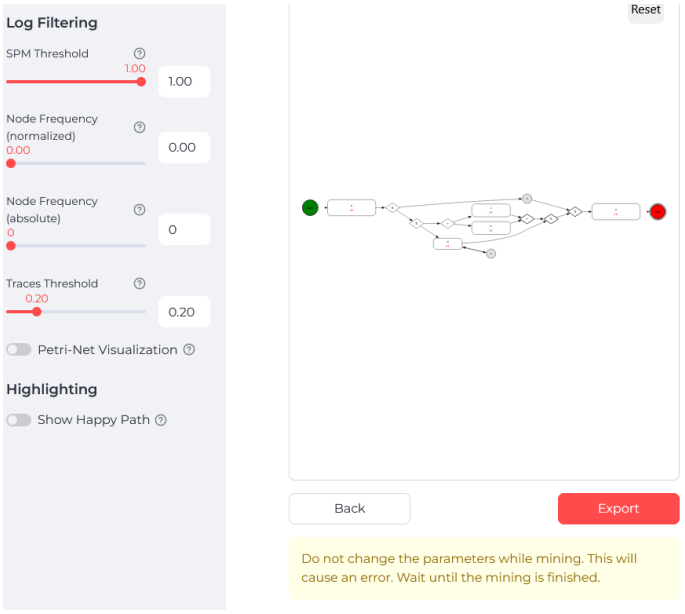
The existing unit tests for Inductive Miner Infrequent from the previous student project[1] were migrated and adapted. The test scenarios were preserved while imports, mining parameters, process-tree utilities, and logging integration were adapted to match the current project structure and interfaces. The test case files were also added for manual testing.

4.2.3 BPMN Graph Edge Case

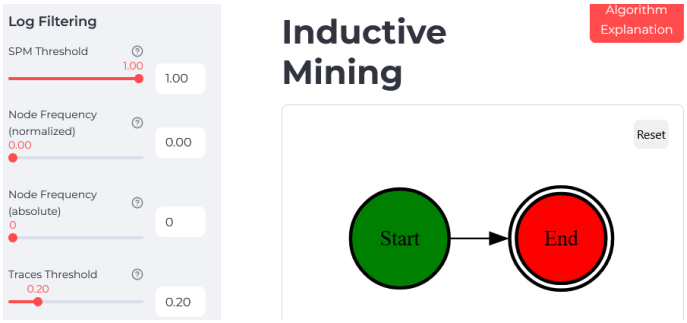
The BPMN graph creation fix was evaluated by reproducing the edge case that originally caused the error. For the Inductive Miner, the SPM threshold was set to `1.0`, which filters all activities from the resulting process model. Before the fix, the application was unable to generate the expected BPMN graph and displayed an error instead. After the implementation was changed, the same configuration successfully produced a valid BPMN graph containing only the Start and End nodes connected by an edge. Figure 4.1 shows the application behavior before and after the fix.

4.2.4 Graph Layout Feature

The static graph layout functionality was tested at the `BaseGraph` level. One test verifies that an edge which is present in the stored reference graph but is missing in newly generated graph is set as invisible. An additional test covers node identifiers that require quoting in Graphviz, for example identifiers containing spaces. It verifies that filtered nodes and their connected edges are correctly marked as invisible while producing a valid Graphviz declarations. The feature was also evaluated manually through the user interface by comparing the positions of graph elements for different filter values.



(a) before fix



(b) after fix

Figure 4.1: BPMN graph generation before and after the fix when all activities are filtered out.

5 Conclusion and Future Work

5.1 Conclusion

The ProcessIntel project was extended with the Inductive Miner Infrequent process mining algorithm and support for the XES file format, including file upload, download, and conversion. The output of the CSV to XES converter was additionally validated by comparison with PM4Py. Several existing bugs were fixed, including column-merging issue during CSV to XES conversion and BPMN graph generation issue when all activities are filtered out. Moreover, a new layout feature was implemented to preserve the positioning of graph elements when filtering parameters are changed.

5.2 Limitations

The static graph layout for now only works for DFG-based algorithms, specifically the Heuristic Miner and Fuzzy Miner. Supporting BPMN graphs and Petri nets is more complex because filtering can change paths and the overall graph structure, which may lead to inconsistent layouts or incorrect graph properties. Another limitation is that edge weights are not recalculated while the static graph layout is enabled. As a result, after nodes are filtered out, the remaining edges may keep their previous weights until the static layout is disabled and the graph is regenerated.

Another limitation concerns performance when processing large event logs. Conversion of large XES files can require considerable time during mining. For example, the `BPI_Challenge_2012.xes` file, with a size of 70MB, required approximately 65 seconds to process in the tested environment.

The evaluation using PM4Py was performed only for the conversion from CSV to XES and for selected test input data. Consequently, this evaluation does not cover all possible event log structures or the reverse conversion from XES to CSV.

5.3 Use of AI Tools

During the development of this thesis, AI was used as a supporting tool for both software development and research. During the implementation, it was mainly used to learn Python and Git rebasing, as well as to clarify programming concepts to meet the requirements of this project. It was also used to create some sample data similar to the BPI Challenge 2013 dataset [10], which was used in unit tests to confirm the resolution of the column-merging bug in CSV to XES conversion. In addition, AI was used to learn about Streamlit's `session_state` and the behavior of the view controller structure. For coding-related

questions and corrections, ChatGPT[1] was used. For researching and reviewing theoretical concepts related to process mining algorithms and event logs, Anara[2] was used.

5.4 Lessons Learned

Working on an existing open-source project is quite different from working on something of one's own. This thesis showed me that difference. Adding new features and fixing bugs had to follow strict project requirements and coding conventions in order to remain maintainable and compatible with other parts of the application. Adapting code from a previous student project to the existing architecture required code comprehension, an understanding of the architecture of both projects, and code refactoring and adaptation. To implement that, I applied theoretical knowledge gained during my studies, including different engineering principles and design patterns. At the beginning of the project, I had little to no experience with Python, Streamlit, XES and CSV formats, or Graphviz. During the thesis, I gained practical experience working with these technologies and tools. I also gained practical experience with Git workflows, including rebasing, resolving conflicts, maintaining a clean commit history, and preparing changes for review.

5.5 Future Work

Future work should address limitations identified during the implementation and evaluation of this thesis. The static layout could be extended to BPMN graphs and Petri nets, although filtering can cause larger structural changes in these graph types. Another improvement would be to assign correct edge weights while preserving the static graph layout. The evaluation of XES conversion could also be extended. The current PM4Py comparison covers only the CSV to XES conversion, the XES to CSV conversion is not yet validated. Further work could also cover performance improvements for processing and mining large event logs. Some of these improvements are already documented as open issues in the ProcessIntel repository, providing concrete directions for future development[3].

[1] ChatGPT: `https://chatgpt.com/`.
[2] Anara: `https://anara.com`.
[3] ProcessIntel issue tracker: `https://github.com/SWISDATA/processintel/issues`.

Bibliography

[1] Florian Eckhard, Holger Wittges, and Stefanie Rinderle-Ma. Exploring the role of visualization in process mining: A systematic literature review. In Krzysztof Wecel, editor, *Business Information Systems*, volume 554 of *Lecture Notes in Business Information Processing*, pages 165–178. Springer, 2025.

[2] Graphviz. Graphviz. `https://graphviz.org/`. Accessed: 2026-09-03.

[3] Christian W. Günther and H. M. W. Verbeek. XES – standard definition. Technical Report 1409, BPMcenter.org, 2014.

[4] IEEE. IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams. Technical Report IEEE Std 1849-2023, IEEE, 2023.

[5] IEEE Task Force on Process Mining. About xes. `https://tfpm.compute.dtu.dk/resources/xes-standard/about-xes`. Accessed: 2026-09-07.

[6] Simone Kriglstein, Margit Pohl, Stefanie Rinderle-Ma, and Magdalena Stallinger. Visual analytics in process mining: Classification of process mining techniques. In Natalia Andrienko and Michael Sedlmair, editors, *EuroVA 2016 - EuroVis Workshop on Visual Analytics*, pages 43–47. The Eurographics Association, 2016.

[7] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. Discovering block-structured process models from event logs containing infrequent behaviour. In *Business Process Management Workshops*, volume 171 of *Lecture Notes in Business Information Processing*, pages 66–78. Springer, 2014.

[8] pandas development team. pandas. `https://pandas.pydata.org/`. Accessed: 2026-09-06.

[9] Process Intelligence Solutions GmbH. PM4Py – process mining for python. `https://processintelligence.solutions/pm4py`. Accessed: 2026-09-06.

[10] Ward Steeman. BPI Challenge 2013, Closed Problems. `https://doi.org/10.4121/uuid:c2c3b154-ab26-4b31-a0e8-8f2350ddac11`, 2013. Dataset, Version 1.

[11] Streamlit. Streamlit. `https://streamlit.io/`, 2026. Accessed: 2026-08-28.

[12] Wil M. P. van der Aalst. *Process Mining: Data Science in Action*. Springer, Berlin, Heidelberg, 2 edition, 2016.

[13] Wil M. P. van der Aalst. Foundations of process discovery. In Wil M. P. van der Aalst and Josep Carmona, editors, *Process Mining Handbook*, volume 448 of *Lecture Notes in Business Information Processing*, pages 37–75. Springer, Cham, 2022.

[14] Wil M. P. van der Aalst. Process mining: A 360 degree overview. In Wil M. P. van der Aalst and Josep Carmona, editors, *Process Mining Handbook*, volume 448 of *Lecture Notes in Business Information Processing*, pages 3–34. Springer, Cham, 2022.